

Model optimization strategies

Most of the LLMs used nowadays, like GPT or Llama, are powered by a decoder-only Transformer architecture. The *decoder-only* architecture is designed for text-generation tasks. It predicts the next word in a sequence based on preceding words, making it effective for generating contextually appropriate text continuations.

In contrast, an *encoder-only* architecture, like BERT, focuses on understanding and representing the input text with detailed embeddings. It excels in tasks that require comprehensive context understanding, such as text classification and named entity recognition. Finally, the encoder-decoder architecture, like T5, combines both functionalities. The encoder processes the input text to generate a context-rich representation, which the decoder then uses to produce the output text. This dual structure is particularly powerful for sequence-to-sequence tasks like translation and summarization, where understanding the input context and generating a relevant output are equally important.

In this book, we only focus on the decoder-only architecture, which dominates the LLM field.

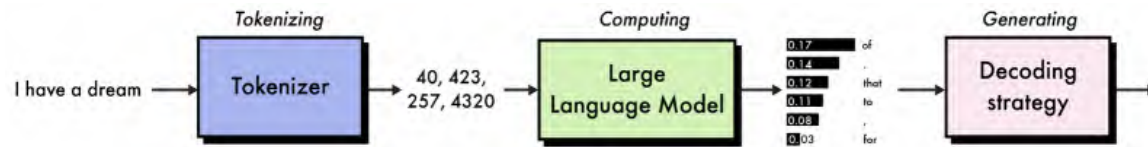


Figure 8.1 – Inference process with decoder-only models. We provide “I have a dream” as input and obtain “of” as output.

As shown in *Figure 8.1*, the basic inference process for a decoder-only model involves:

1. **Tokenizing** the input prompt and passing it through an embedding layer and positional encoding.
2. **Computing** key and value pairs for each input token using the multi-head attention mechanism.
3. **Generating** output tokens sequentially, one at a time, using the computed keys and values.

While *Steps 1* and *2* are computationally expensive, they consist of highly parallelizable matrix multiplication that can achieve high hardware utilization on accelerators like GPUs and TPUs.

The real challenge is that the token generation in *Step 3* is inherently sequential – to generate the next token, you need to have generated all previous tokens. This leads to an iterative process where the output sequence is grown one token at a time, failing to leverage the parallel computing capabilities of the hardware. Addressing this bottleneck is one of the core focuses of inference optimization.

In this section, we will detail several optimization strategies that are commonly used to speed up inference and reduce **Video Random-Access Memory (VRAM)** usage, such as implementing a (static) KV cache, continuous batching, speculative decoding, and optimized attention mechanisms.

KV cache

We saw that LLMs generate text token by token, which is slow because each new prediction depends on the entire previous context. For example, to predict the 100th token in a sequence, the model needs the context of tokens 1 through 99. When predicting the 101st token, it again needs the information from tokens 1 through 99, plus token 100. This repeated computation is particularly inefficient.

The **key-value (KV)** cache addresses this issue by storing key-value pairs produced by self-attention layers. Instead of recalculating these pairs for each new token, the model retrieves them from the cache, significantly speeding up the generation.

You can see an illustration of this technique in *Figure 8.2*:

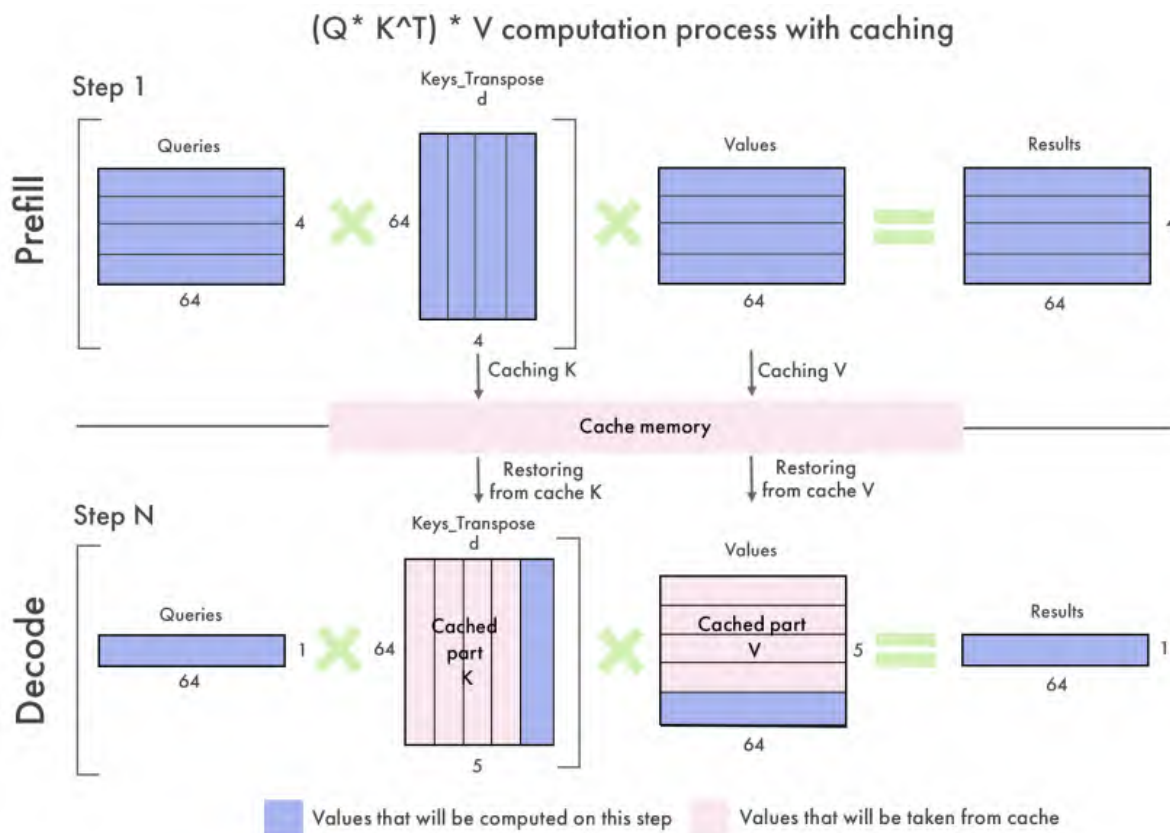


Figure 8.2 – Illustration of the KV cache

When a new token is generated, only the key and value for that single token need to be computed and added to the cache. The KV cache is an immediate optimization that is implemented in every popular tool and library. Some implementations maintain a separate KV cache for each layer of the model.

The size of the KV cache scales with the number of tokens (n_{tokens}) and several model dimensions, like the number of layers (n_{layers}), the number of attention heads (n_{heads}), their dimension ($\dim_{head}$), and the precision of the parameters in bytes (n_{bytes}):

$$size(KV\ cache) = 2n_{tokens} n_{layers} n_{heads} \dim_{head} n_{bytes}$$

For a typical 7B parameter model using 16-bit precision, this exceeds 2 GB for high sequence lengths (higher than 2,048 tokens). Larger models with more layers and higher embedding dimensions will see even greater memory requirements.

Since the KV cache grows with each generation step and is dynamic, it prevents you from taking advantage of `torch.compile`, a powerful optimization tool that fuses PyTorch code into fast and optimized kernels. The *static KV cache* solves this issue by pre-allocating the KV cache size to a maximum value, which allows you to combine it with `torch.compile` for up to a 4x speedup in the forward pass.

To configure a model to use a static KV cache with the transformers library, follow these steps:

1. We import the tokenizer and the model we want to optimize:

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_id = "google/gemma-2b-it"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, device_
map="auto")
```

2. To implement the static cache, we change the cache implementation in the model's generation config to static:

```
model.generation_config.cache_implementation = "static"
```

3. Now that our KV cache is static, we can compile the model using `torch.compile`:

```
compiled_model = torch.compile(model, mode="reduce-overhead",
fullgraph=True)
```

4. We tokenize an input question, "What is 2+2?", and store it on a GPU if available (if not, we store it on the CPU):

```
device = "cuda" if torch.cuda.is_available() else "cpu"
inputs = tokenizer("What is 2+2?", return_tensors="pt").to(device)
```

5. Let's use the `generate()` method to get the model's output and decode it with `batch_decode()` to print its answer:

```
outputs = model.generate(**inputs, do_sample=True, temperature=0.7,
max_new_tokens=64)
print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
['What is 2+2?\n\nThe answer is 4. 2+2 = 4.']
```

This returns a list containing both the input and output, correctly answering our question.



Note that the static cache doesn't work with all architectures. For details on which architectures are supported, check out the transformers documentation.

Efficiently managing the KV cache is essential, as it can quickly exhaust available GPU memory and limit the batch sizes that can be processed. This has motivated the development of memory-efficient attention mechanisms and other techniques, which we will cover in the last section.

Continuous batching

Batching, or processing multiple inference requests simultaneously, is a standard approach to achieve high throughput. Larger batch sizes spread out the memory cost of model weights and transfer more data to the GPU at once, better saturating its parallel compute capacity.

However, decoder-only models pose a particular challenge due to the high variability in input prompt lengths and desired output lengths. Some requests may have short prompts and only need a one-word answer, while others may input a lengthy context and expect a multi-paragraph response.

With traditional batching, we would have to wait for the longest request in a batch to complete before starting a new batch. This leads to under-utilization as the accelerator sits partly idle waiting for a straggling request to finish. *Continuous batching*, also known as in-flight batching, aims to prevent idle time by immediately feeding a new request into the batch as soon as one completes.

The batching process begins the same – by filling the batch with initial requests. But as soon as a request completes its generation, it is evicted from the batch and a new request takes its place. This way, the accelerator is always processing a full batch, leading to maximally efficient hardware utilization. An additional consideration is the need to periodically pause the generation process to run prefill, or the embedding and encoding of waiting requests. Finding the optimal balance between generation and prefill requires some tuning of the waiting-served ratio hyperparameter.

Continuous batching is natively implemented in most inference frameworks, like Hugging Face's **Text Generation Inference (TGI)**, vLLM, and NVIDIA TensorRT-LLM.

Speculative decoding

Another powerful optimization technique is *speculative decoding*, also called assisted generation. The key insight is that even with continuous batching, the token-by-token generation process fails to fully saturate the parallel processing capabilities of the accelerator. Speculative decoding aims to use this spare compute capacity to predict multiple tokens simultaneously, using a smaller proxy model (see Figure 8.3).

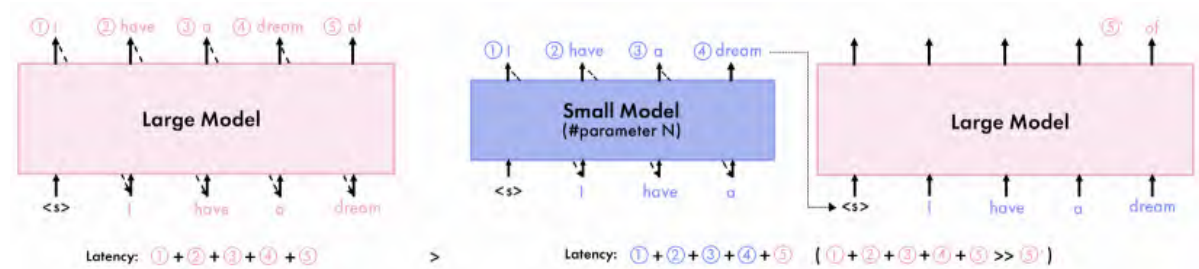


Figure 8.3 – Illustration of traditional decoding (left) and speculative decoding (right)

The general approach is:

- Apply a smaller model, like a distilled or pruned version of the main model, to predict multiple token completions in parallel. This could be 5–10 tokens predicted in a single step.
- Feed these speculative completions into the full model to validate which predictions match what the large model would have generated.
- Retain the longest matching prefix from the speculative completions and discard any incorrect tokens.

The result is that, if the small model approximates the large model well, multiple tokens can be generated in a single step. This avoids running the expensive large model for several iterations. The degree of speedup depends on the quality of the small model's predictions – a 90% match could result in a 3–4X speedup.

It is crucial that both models use the same tokenizer. If this is not the case, the tokens generated by the draft model will not align with those produced by the large model, making them incompatible. Let's implement this using the transformers library. In this example, we will use two Qwen1.5 models from Alibaba Cloud: a 1.8B version as the main model, and a 0.5B version as the draft model. Note that, if you have enough VRAM, you can use much larger models like 14B, 32B, 72B, or 110B as the main model.

Here, we're limited by the VRAM of the T4 GPU in Google Colab, but to get the maximum speedup, the assistant model should be much smaller than the large model.

Here's a step-by-step guide to implement speculative decoding:

1. We load the tokenizer and both models:

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_id = "Qwen/Qwen1.5-1.8B-Chat"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto")
draft_model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen1.5-0.5B-Chat", device_map="auto")
```

2. We then tokenize the same input and store it in the accelerator, if available:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
inputs = tokenizer("What is 2+2?", return_tensors="pt").to(device)
```

3. We can now use `model.generate()` with the argument `assistant_model=draft_model` to enable speculative decoding:

```
outputs = model.generate(**inputs, do_sample=True, assistant_model=draft_model, temperature=0.7, max_new_tokens=64)
print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
['What is 2+2? 2 + 2 equals 4!']
```

The speedup in this small example is not significant, but it is clearly noticeable with bigger models.

Prompt lookup decoding is a variant of speculative decoding, tailored to input-grounded tasks like summarization where there is often overlap between the prompt and output. Shared n-grams are used as the LLM candidate tokens. We can enable prompt lookup decoding by using the `prompt_lookup_num_tokens` parameter in `model.generate()`:

```
outputs = model.generate(**inputs, prompt_lookup_num_tokens=4)
```

By combining the static KV cache with `torch.compile`, implementing continuous batching, and leveraging speculative decoding techniques, LLMs can see inference speedups of 2–4x or more with no loss in quality.

Another approach to creating a small proxy model consists of jointly fine-tuning a small model alongside a large model for maximum fidelity. A representative technique here is Medusa, which inserts dedicated speculation heads into the main model. The Medusa-1 approach fine-tunes these speculation heads while freezing the large model, while the Medusa-2 approach jointly fine-tunes both the speculation heads and the large model. The Medusa method has demonstrated impressive results, enabling a 70M parameter model to closely approximate the performance of a 7B parameter model on a range of tasks. Speculative decoding is natively supported by TGI.

Optimized attention mechanisms

The Transformer architecture is based on the attention mechanism, which scales quadratically with the number of input tokens (or sequence length). This is particularly inefficient for longer sequences, where the size of the KV cache can blow up.

Introduced by Kwon, Li, et al. (2023), *PagedAttention* addresses these memory challenges by drawing inspiration from virtual memory and paging in operating systems. It partitions the KV cache into blocks, eliminating the need for contiguous memory allocation. Each block contains the keys and values for a fixed number of tokens. During attention computation, the PagedAttention kernel efficiently fetches these blocks, regardless of their physical memory location.

This partitioning allows for near-optimal memory utilization. This is useful for batching more sequences together, which increases throughput and GPU utilization. Moreover, PagedAttention's block-based approach naturally supports memory sharing across multiple output sequences generated from the same prompt. This is particularly advantageous in parallel sampling and beam search, where the same prompt is used to generate multiple outputs. The shared memory blocks reduce redundant computations and memory usage, cutting the memory overhead by up to 55% and improving throughput by up to 2.2x, according to the authors. The vLLM library received the first implementation of PagedAttention. Since then, PagedAttention has also been implemented in TGI and TensorRT-LLM.

Another popular option is *FlashAttention-2*. Developed by Tri Dao (2023), it introduced several key innovations that are designed to address the quadratic runtime and memory constraints in traditional attention. By dividing input and output matrices into smaller blocks, FlashAttention-2 ensures that these blocks can fit into the GPU's on-chip SRAM, which is much faster than high-bandwidth memory. This approach significantly reduces the frequency of data transfers between the GPU's main memory and its processing units.